엔진의 계산 손실은 LLM 의 출력을 다시 굴리지만 실패 증상을 늘린다는 증거는 없다:

8 비트 정밀도에서 계약대로 정확한 기준으로 본 연구

Sunghyeun Woo
청운대학교

영문 초안(arXiv 제출본, cs.CL / cs.LG)의 한국어 번역 · 2026 년 10 월

엔진과 실험: https://github.com/wwoosshh/GeulPU
언어와 컴파일러: https://github.com/wwoosshh/geul-lang

요약

대규모 언어 모델은 때때로 익숙한 실패를 낸다 — 끝없는 반복, 엉뚱한 문자로 쓴 글, 깨진 맞춤법, 틀린 답. 한 가설은 이 실패의 일부를 추론 엔진의 계산 손실 탓으로 돌린다 — 8 비트로 다시 양자화한 활성값, 반정밀도의 어텐션과 캐시, 묶음에 따라 달라지는 덧셈 차례. 이를 시험하려면 합쳐 보면 똑같아 보이는 두 효과를 갈라야 한다: 아슬아슬한 결정의 주사위를 *다시 굴리기만* 하는 손실과, 모델을 비정상 토큰 쪽으로 체계적으로 *기울이는* 손실이다. 우리는 GeulPU — 산술이 적힌 정밀도 계약을 따르고, CPU 참조 구현과 비트까지 같으며, GPU 세대와 제조사가 달라도 같은 비트를 내는 추론 엔진 — 를 계약대로 정확하고(contract-exact) 잡음 없는 기준으로 삼아, llama.cpp 그리고 손실 하나만 넣은 변형과 견주었다(Qwen2.5-7B-Instruct 와 DeepSeek-R1-Distill-Qwen-7B, Q8_0, 탐욕 디코딩, 같은 토큰 입력). 한국어 질문 696 개와 수학 문제 600 개에서 llama.cpp 는 답의 89–100 % 를 바꾸었지만, 증상은 생기는 만큼 사라졌다: 다른 문자의 글 생김 36 · 사라짐 35, 틀렸거나 끝내지 못한 수학 답 25 · 28 — 순증가의 95 % 구간은 기준 쪽 증상 수의 −17–+19 % 와 −22–+15 % 안이다. 비정상 토큰이 고른 토큰과 아슬아슬하게 겨루는 위험한 걸음 10,817 개에서 다른 문자 · 글자 조각 쪽으로의 기울기는 없었고(±0.05 nat 안의 같음), 되풀이 쪽으로 작은 기울기(+0.0039 nat, 승산으로 약 0.4 %)가 있었는데 이는 되풀이 토큰에만 특별했다. 탐욕 디코딩과 계약대로 정확한 계산에서 증상의 대부분은 모델 자신에게서 나오고(긴 한국어 글의 32 % 가 길이 한도까지 되풀이한다), 다른 문자 토큰이 이미 아슬아슬한 경쟁자인 걸음에서 그 토큰을 억지로 넣으면 이어지는 글에 다른 문자가 또 나오는 비율이 80 % 였고, 원래 토큰 뒤에는 34 % 였다. 모든 가설 · 통계 · 판정 규칙은 미리 등록했다.

1 들어가며

대규모 언어 모델(LLM)을 쓰는 사람은 되풀이되는 몇 가지 실패를 안다: 모델이 토큰이 바닥날 때까지 한 구절을 반복하고, 답 한가운데서 엉뚱한 언어나 문자로 바꾸고, 조금 전에 바르게 쓴 낱말을 틀리게 쓰고, 문제의 요점을 잃는다. 이 실패는 보통 모델 탓으로 돌려진다 — 훈련 자료, 크기, 디코딩 설정. 경쟁하는 가설은 그 일부가 추론 엔진의 산술에서 온다는 것이다. 실제 엔진은 여러 곳에서 정밀도를 속도와 바꾼다: llama.cpp 는 양자화된 행렬 곱 앞에서 활성값을 8 비트로 다시 양자화하고, 어텐션을 반정밀도로 계산하며 키-값 캐시도 반정밀도로 저장하고, 대부분의 엔진이 그렇듯 묶음의 구성에 따라 부동소수점 덧셈의 차례를 바꾼다. 이 선택 하나하나가 로짓을 조금씩 흔들고, 아슬아슬한 자리에서 조금 다른 로짓은 생성을 다른 길로 보낼 수 있다.

엔진의 계산이 출력을 바꾼다는 것은 이미 잘 알려져 있다. 묶음 크기 · GPU 수 · 엔진만 바꿔도 벤치마크 점수가 몇 점씩 움직이고 (Yuan et al., 2025; Pape et al., 2026), 접두 캐시는 같은 요청을 갈라지게 하며 (Patodiya, 2026), 한 기계 안의 변동을 없애는 결정 모드가 만들어졌다 (He et al., 2025; Qin et al., 2026). 알려지지 않은 것은 이 변동이 출력을 *망가뜨리는가* — 실패를 더 만드는가 — 아니면 *주사위를 다시 굴릴 뿐*인가 — 어떤 답은 나빠지고 그만큼은 좋아지는가 —

다. 둘은 한 번의 실행에서는 구별할 수 없다. 둘이 갈리는 곳은 생기는 실패와 사라지는 실패의 균형이고, 결정 하나하나의 수준에서는 흔들림이 비정상 토큰 쪽으로 치우쳐 있는가다.

그 치우침을 재려면 엔진 잡음이 없는 기준이 필요하다. 기준 자체에 잡음이 있으면 비정상 토큰이 고른 토큰에 가까운 결정을 고르는 일 자체가 잡음에 달려, 재어지는 이동은 0 쪽으로 눌린다. 그래서 우리는 손실이 적힌 계약이 정한 만큼뿐이고 비트가 묶음 · 길 · GPU · 제조사에 달려 있지 않은 엔진을 기준으로 삼고(3절), 모든 엔진에 같은 토큰 번호를 넣고, 분석을 미리 등록했다.

이 글의 기여는 다음과 같다.

1. 계약대로 정확한 기준을 둔 짝 설계로 *다시 굴리기*(증상이 같은 수만큼 생기고 사라진다)와 *기울이기*(비정상 토큰 쪽으로의 체계적 이동)를 가른다. 미리 등록한 1차 시험은 둘이다 — 순증가의 구간을 함께 적는 궤적 수준의 McNemar 검정과, 같음 띠를 둔 위험한 걸음의 걸음 수준 기울기 시험.
2. Q8_0 · 탐욕 디코딩의 7B 모델 둘에서 llama.cpp 의 계산 손실은 출력을 다시 굴리되 실패를 늘린다고 잡히지 않으며 — 순증가는 기준 쪽 증상 수의 약 ±20 % 안에 묶인다 — 잡히는 유일한 기울기는 되풀이 쪽으로 승산 약 0.4 % 라는 증거.
3. 계약대로 정확한 계산과 탐욕 디코딩에서 잰 모델 수준의 측정: 높은 증상 바탕 비율, 이미 그럴듯한 자리에 넣은 다른 문자 토큰 하나의 효과, 앞 차례에 넣은 비정상 하나의 효과 없음, 그리고 바이트 조각으로 나오는 한국어 음절의 비율.
4. 측정 도구 자체: 적힌 정밀도 계약을 가진 공개 추론 엔진으로, NVIDIA 세대 · AMD · Intel GPU · CPU 참조 구현에서 출력 비트가 같다(3절, 부록 A · 부록 D).

2 관련 연구

엔진의 비결정성과 평가. He et al. (2025) 는 서비스되는 LLM 의 실행마다의 차이를 묶음에 따라 달라지는 커널에서 찾아 묶음 불변 커널을 내놓았고, vLLM 과 SGLang 은 그에 바탕한 결정 모드를 실었다. Vosti 는 결정적 추론을 형식적으로 명세하고 엔진을 그 명세에 대해 검증하여, 실제 엔진의 결정 모드가 어떤 실행 변화에서는 여전히 갈라짐을 보였다 (Qin et al., 2026). Yuan et al. (2025) 는 DeepSeek-R1-Distill-Qwen-7B 의 정확도가 GPU 수 · GPU 종류 · 묶음 크기에 따라 bfloat16 에서 최대 9 % 흔들린다고 보고했고, Pape et al. (2026) 는 엔진 선택만으로 벤치마크 점수가 최대 16.6 점 움직임을 보였으며, Patodiya (2026) 는 llama.cpp 의 접두 캐시가 되풀이한 요청을 갈라지게 하고 4 비트에서 더함을 보였다. 이 연구들은 분산이 있음을 세웠다. 우리는 그 분산이 치우쳐 있는가를 묻는다.

하드웨어를 가로지르는 비트 정확 추론. CPU 에뮬레이터로 고치지 않은 vLLM · Transformers 가 여러 NVIDIA GPU 에서 내는 비트를 예측하여, 엔진을 바꾸지 않고 검증할 수 있다 (arXiv:2606.00279, 2026). 고정된 모양의 덧셈 차례는 선형층을 NVIDIA Ampere · Ada · Hopper 에서 같게 만든다 (Cooper et al., 2026). 2 의 거듭제곱 가중치 스케일은 CUTLASS 와 Triton 을 비트까지 일치시키고, 허용 오차에 기댄 적합성 시험의 한계를 드러낸다 (Chen, 2026). Gensyn 의 REE 는 비공개 연산자 라이브러리로 NVIDIA GPU · Apple GPU · CPU 에서 추론을 재현한다. 우리가 아는 한 NVIDIA 와 AMD GPU 에서 같은 비트를 공개된 적힌 정밀도 계약과 함께 보인 발표된 시스템은 없다. 우리는 그런 시스템을 연구 대상이 아니라 측정 도구로 쓴다.

실패 양상. 언어 혼동 벤치마크(Language Confusion Benchmark)는 모델이 엉뚱한 언어로 답하는 빈도를 잰다 (Marchisio et al., 2024a). 양자화는 비라틴 문자 언어를 가장 크게 해치고, 자동 지표는 그 해를 작게 본다 (Marchisio et al., 2024b). 반복은 스스로를 강화하고 — 반복할수록 다음 반복이 더 쉬워진다 (Xu et al., 2022) — 반복 요청은 실제 서비스 모델을 엇나가게 할 수 있다 (Nasr et al., 2023). 덜 배운 "글리치" 토큰은 비정상 행동을 일으킨다 (Land and Bartolo, 2024). 계산 손실의 원천을 하나씩 넣거나 빼고 계약대로 정확한 기준으로 이 실패들에의 효과를 잰 연구는 우리가 아는 한 없다.

3 측정 도구: GeulPU

우리의 결론은 기준의 두 성질에 기댄다: 그것이 llama.cpp 와 같은 양자화 모델 파일을 같은 GPU 에서 돌리는 완전한 추론 엔진이라는 것, 그리고 그 산술이 적힌 계약에 대해 정확하다는 것이다. 둘 다 Python · PyTorch · CUDA 툴킷 위에 세워지지 않은 소프트웨어로서는 드문 일이고, 그것을 쓴 언어는 거의 알려져 있지 않다. 부록 D 가 언어 · 컴파일러 · 엔진을 바닥부터 설명하고 거기의 그림 1 이 전체 구조를 보인다. 이 절은 실험이 기대는 것만 적는다.

3.1 엔진 간추림

GeulPU 는 글(geul) — 문법이 한국어를 따르는 시스템 언어로, 자기 자신을 컴파일하는 컴파일러가 자기 x86-64 백엔드를 가지며 C 컴파일러도 LLVM 도 쓰지 않는다 — 로 쓰였다. 역시 글로 쓴 커널 생성기가 모델 전용 GPU 커널을 쓰고, 글로 쓴 두 백엔드가 그것을 PTX(NVIDIA)와 SPIR-V(Vulkan: AMD · Intel · NVIDIA)로 옮긴다. 엔진은 PTX 나 SPIR-V 를 제조사 드라이버에 넘기고 드라이버하고만 말한다: CUDA 툴킷이나 Vulkan SDK 의 어느 것도 빌드와 실행에 쓰이지 않는다. 호스트 — GGUF 읽기, 토크나이저, 스케줄러, 샘플러, 최대 8 슬롯의 OpenAI 호환 HTTP 서버 — 는 글 약 13,000 줄이다. llama.cpp 와 같은 GGUF 파일(Q8_0 · Q4_0 · k-quant; Qwen2.5 · Qwen3 · DeepSeek-R1-Distill-Qwen · Llama 3.1/3.2 · Mistral)을 읽고, RTX 4070 Ti 에서 Q8_0 · Q4_K_M 의 디코딩 속도는 llama.cpp 의 ±5 % 안이다. 계약의 CPU 참조 구현, 시험 도구, 벤치마크는 Python(NumPy)이다.

3.2 정밀도 계약

GeulPU 의 산술은 하드웨어에 재량을 남기지 않는 적힌 계약(`docs/contract.md`)을 따른다.

- **선형층.** 활성값 32 개 묶음마다 가장 큰 값에서 정한 2 의 거듭제곱으로 스케일하고 최대 2^{14} 의 정수로 반올림하여, 부호 있는 8 비트 자리 둘로 저장한다. 모델의 정수 가중치와의 곱은 int8 텐서 코어(프롬프트)나 `dp4a`(디코딩)로 정확히 더하고 — 정확한 정수 덧셈은 차례에 달려 있지 않다 — 묶음마다 한 번 반올림한 뒤 고정된 차례의 융합 곱셈-덧셈으로 누적한다.
- **어텐션.** 쿼리 · 키 · 값도 같은 방식으로 바꾸고 키-값 캐시는 그 자리들을 저장한다. 점수는 한 번 반올림한 정확한 정수 합이고, 지수 함수는 적힌 다항식이며, 키는 고정된 64 키 덩어리로 처리하여 고정된 두 단계 차례로 접는다.
- **그 밖의 모든 것.** 정규화는 고정된 덧셈 나무를 쓰고, 회전 표는 바르게 반올림되며, 비정규 수를 어느 길에서나 어떻게 0 으로 보내는지를 규칙 하나가 정한다(PTX `.ftz`, Vulkan 드라이버들, CPU 참조 구현이 이에 일치한다). 샘플링과 로그 확률은 float64 로 고정된 차례로 계산한다.

일을 어떻게 나누는지에 아무것도 달려 있지 않으므로, 비트는 묶음 · 슬롯 · 덩어리 나누기 · 프롬프트인지 디코드인지의 길 · GPU · 제조사에 달려 있지 않고, NumPy 로 쓴 계약의 CPU 참조 구현의 비트와 같다. 이런 기준을 *계약대로 정확하다(contract-exact)* 고 부른다: 계산 손실은 계약이 적어 둔 것뿐이고 결정적이다 — 그러나 정확한 산술은 아니다. 계약의 손실은 작다: Qwen2.5-0.5B(Q8_0)에서 다음 토큰 분포의 fp64 계산에 대한 쿨백-라이블러 발산은 2.24×10^{-6} 이고 llama.cpp 는 1.38×10^{-3} 이다. 7B · 14B 의 해당 비(1/3,805 · 1/4,467)는 엔진이 아니라 반올림 지점의 시뮬레이션에서 나온 값이다. 계약의 반올림 자체가 fp64 대비 위험한 걸음을 기울이는지는 Qwen2.5-0.5B 에서 부록 E 로 잰다 *[측정 중]*. 정확함에는 값이 있다: 같은 최적화에서 활성값 자리를 하나가 아니라 둘로 두면 프롬프트 처리가 1.26–1.38 배 느려지고(타일 커널을 바꿔 잰 하한), 메모리에 묶인 디코딩은 영향이 없다.

3.3 비트 동일성

계약의 주장은 커널 시험 사례, 모델 전체의 끝에서 끝까지, 실행 길 사이, 그리고 기계들 사이에서 확인한다: CUDA 로 NVIDIA Ampere · Ada · Blackwell GPU 에서, Vulkan 으로 NVIDIA 하나와 AMD 둘에서, 그리고 CPU 참조 구현에 대해 모든 걸음의 로짓 해시가 같다(부록 A). PTX

표 1: 질문 묶음.

묶음	수	최대 새 토큰	모델	출처
자연: 한국어 LCB	396	1,024	Qwen2.5	언어 혼동 벤치마크 (Marchisio et al., 2024a) 의 한국어 native · 교차 언어 질문 (Apache-2.0); 너무 긴 질문 3 개 제외
자연: 한국어 긴 글	300	2,048	Qwen2.5	주제 60 × 꼴 5("2,000자 이상으로 써라")
자연: 수학	600	3,200	R1-Distill	gsm8k test (Cobbe et al., 2021) 앞 600, "Please reason step by step, and put your final answer within `\boxed{}`." 를 붙임
방아쇠: 희귀 음절	200	512	Qwen2.5	"다음 글을 그대로 옮겨 써라": 토크나이저가 바이트 조각으로 쪼개는 음절이 6 개 이상 든 80–200 자의 글
방아쇠: 섞인 문자	200	1,024	Qwen2.5	중국어 · 일본어 낱말과 이름을 그 문자 그대로 적은 한국어 질문
방아쇠: 되풀이	200	2,048	Qwen2.5	낱말이나 문장을 되풀이하라는 요청, 한글로 수세기, 무늬 이어 쓰기
오염 대화	100 × 4	512 + 1,024	Qwen2.5	두 차례 대화; 첫 답은 G 자신의 답을 그대로 두거나 비정상 하나를 넣은 것

와 SPIR-V 아래의 제조사 비공개 컴파일러는 계약 밖이라, 비트 동일성은 증명이 아니라 실측으로 세운다.

4 방법

4.1 갈래와 입력

- **G (계약대로 정확한 기준)** — 정밀도 계약 아래의 GeulPU(3절): 손실은 계약이 적은 곳뿐, 묶음 · 슬롯 · 실행 길에 달려 있지 않음. OpenAI 호환 서버, 슬롯 8.
- **L (실제 엔진)** — llama.cpp 빌드 b11669(Windows CUDA 13.4), 기본 설정의 `llama-server` (플래시 어텐션 *auto*, F16 키-값 캐시, 프롬프트 캐시 켬), 병렬 슬롯 8, 같은 GGUF 파일.
- **V (손실 하나)** — G 에서 하나만 바꾼 것: 모든 선형층의 입력 활성값을 두 자리가 아니라 한 자리로 반올림 — 묶음 격자 2^{E-5}, $|n| \le 64$, 곧 2 의 거듭제곱 스케일에 값마다 7 비트. llama.cpp 의 활성값 양자화(Q8_1: 32 개 묶음을 max /127 로 스케일)는 정확한 최댓값 스케일에 8 비트라, V 의 반올림 간격은 llama.cpp 의 2–4 배다: V 는 같은 계열의 더 거친 손실이지 llama.cpp 손실의 복제가 아니다. 어텐션과 그 밖은 그대로.

모든 질문은 llama.cpp 의 `/apply-template`(GGUF 에 저장된 대화 틀)로 한 번 만들어 `/tokenize` 로 토큰으로 바꾸었고, 모든 갈래가 같은 토큰 번호를 받았다. 디코딩은 탐욕(온도 0)이고 최대 길이와 끝 토큰이 같아, 갈래 사이의 차이는 산술뿐이다. 문맥은 슬롯마다 4,096 토큰이다. 모델: Qwen2.5-7B-Instruct(한국어 질문)와 DeepSeek-R1-Distill-Qwen-7B(수학; 아래 R1-Distill), 둘 다 Q8_0 이라 가중치의 손실은 갈래 사이에 같고 작다.

4.2 질문 묶음

표 1 가 질문 묶음이다: 자연스러운 질문과, 증상마다 연구할 만큼 자주 나게 지은 방아쇠 묶음.

4.3 증상

모든 증상은 토큰 번호와 글에서 자동으로 판정한다.

- **S1 끝없는 반복** — 생성이 길이 한도에 닿았고 끝이 정확한 되풀이다: 되풀이가 끝의 $\max(3p, 64)$ 토큰 이상을 덮는 가장 작은 주기 $p \le 512$.

- **S2 다른 문자의 글**(한국어 묶음) — S2a: 한글도 라틴도 아닌 글자(한자 · 가나 · 키릴 · 타이 · 아랍 · 데바나가리 등; 그리스 문자 제외)가 두 자 이상 이어진 곳이, 코드 밖에, 질문에 없는 글자로 있다. 또는 S2b: 글자 가운데 한글이 50 % 미만이다.
- **S3 끝내지 못함 / S4 틀린 답**(수학) — S3: `\boxed{}` 없이 길이 한도. S4: 마지막 boxed 의 수가 정답과 다르다(S3 은 틀림으로 센다).
- **S6 맞춤법 깨짐**(옮겨 쓰기 방아쇠) — 옮겨 쓴 부분이 원문과 한 글자라도 다르거나, 깨진 UTF-8 글자가 있다.

4.4 위험한 걸음과 기울기

G 의 생성 걸음마다 a 를 고른 토큰(탐욕: G 의 1위), b 를 G 의 상위 20 가운데 그 유형의 비정상 후보 중 가장 높은 것으로 둔다.

- **O (다른 문자)** — a 는 한글 · 라틴 · 숫자 · 기호이고, b 는 다른 문자의 글자를 담는다 (바이트 조각 토큰은 첫 바이트가 가리키는 문자 범위로 분류한다).
- **R (되풀이)** — a 를 붙이면 답 안의 앞선 8 토큰 묶음을 베끼지 않고, b 를 붙이면 베낀다.
- **C (글자 조각)** — 걸음이 여러 바이트 글자의 한가운데이고(앞 토큰들이 UTF-8 열을 다 채우지 않았다), b 는 다른 글자를 완성하거나 인코딩을 깬다.

여유 $m = \ln p_G(a) - \ln p_G(b)$ 가 $\ln 10$ 보다 작으면 그 걸음은 *위험한 걸음*이다. 갈래 $X \in \{L, V\}$ 에서 그 걸음의 기울기는

$$\Delta_X = \big[\ln p_X(b) - \ln p_X(a)\big] - \big[\ln p_G(b) - \ln p_G(a)\big] \tag{1}$$

이고, 같은 접두(G 의 답)에서 계산한다. 평균 0 의 계산 잡음이면 $\mathbb{E}[\Delta] = 0$ 이고, 비정상 토큰 쪽으로의 체계적 이동이면 $\Delta > 0$ 이다. 대조로 같은 걸음에서 가장 높은 *정상* 대안에 대해 같은 양을 계산한다. 평균 Δ 는 모든 위험한 걸음에서 추정하고, 95 % 구간은 질문 단위로 다시 뽑는 붓스트랩(10,000 번, 씨앗 40)으로 내며, 세 유형은 Holm 방법으로 고친다. 판정 이름(미리 등록): *나쁜 쪽으로 기운다*(평균 > 0, 고친 $p < 0.05$), *좋은 쪽으로 기운다*, *기울지 않는다*(고친 $p \geq 0.05$ 이고 구간이 ± 0.05 nat 안), 그 밖이거나 걸음이 200 개 미만이면 *판단 보류*.

G 의 접두에서 L 의 분포를 얻기 위해 llama.cpp 를 C API 로 몬다: 서버가 하듯 프롬프트는 묶음 하나로 계산하고, G 의 답 토큰을 하나씩 넣되 여덟 줄을 한 묶음으로. 이것은 llama.cpp 산술의 한 실현이다 — G 의 토큰 여덟 줄을 한 묶음으로 넣은 것이라, 서버가 L 자신의 답을 생성할 때의 묶음 구성과 같다는 보장은 없다 — 그래서 `llama-server` 자체와 첫 갈림 전의 걸음에서 대조해 확인했다(5.1절). L 의 궤적(5.2절)은 서버 자신의 것이고, 걸음 수준의 기울기만 이 따라가기를 쓴다.

4.5 궤적 · 번짐 · 오염

- **궤적(1차).** 질문마다 증상이 그 갈래에만 있으면 *생김*, G 에만 있으면 *사라짐*. H1: Qwen2.5-7B, 자연 한국어 묶음, S2. H2: R1-Distill, 수학, S4. McNemar 정확 검정, 둘에 Holm. 판정: *늘린다*, *줄인다*, *흔들지만 늘리지 않는다*(생김 + 사라짐 ≥ 40, 고친 $p \geq 0.05$), *판단 보류*(생김 + 사라짐 < 40; H2 는 먼저 다음 600 문항을 한 번만 더한다 — 필요 없었다). 순증가(생김 − 사라짐)의 95 % 구간(불일치 쌍의 이항 분포에서)도 적는다 — 자료가 늘어남과 줄어듦에 두는 한계다.
- **번짐.** 유형마다 위험한 걸음 최대 100 개(씨앗 40)에서 G 에 b 를 억지로 넣고 256 토큰을 이어 쓰게 하여, a 뒤의 G 자신의 이어 쓰기와 견준다: 같은 유형의 증상이 다시 나는가?
- **오염.** 두 차례 대화의 G 의 첫 답에 비정상 하나 — 중국어 낱말, 맞춤법이 틀린 낱말, 같은 문장 한 번 더 — 를 넣고, 둘째 답을 바꾸지 않은 첫 답 뒤에 생성한 둘째 답과 견준다.

표 2: 1차 궤적 시험, llama.cpp(L)와 계약대로 정확한 기준(G). G / L: 갈래마다 증상이 있는 문항 수; 생김: L 에만 증상이 있는 문항; 사라짐: G 에만; 순증가: 생김 − 사라짐과 불일치 쌍의 이항 분포에서 낸 95 % 구간 (G 의 수에 대한 비율도).

시험	문항	G / L	생김	사라짐	순증가, 95 % 구간	Holm p	판정
H1: Qwen2.5-7B, 자연 한국어, 다른 문자(S2)	696	94 / 95	36	35	$-16 \sim +18$ ($-17\,\% \sim +19\,\%$)	1.0	흔들지만 늘리지 않는다
H2: R1-Distill-7B, gsm8k, 틀렸거나 끝내지 못함(S4)	600	80 / 77	25	28	$-18 \sim +12$ ($-22\,\% \sim +15\,\%$)	1.0	흔들지만 늘리지 않는다

4.6 사전 등록

설계 · 가설 · 통계 · 판정 규칙은 2026-10-16 09:28 에 공개 저장소에 커밋했고(커밋 `9190f76`), 첫 실행은 09:51 에 시작했다. 질문 묶음과 증상 판정기는 G 의 Qwen2.5 실행(09:59–10:39)이 도는 동안인 10:00 에 커밋했고, 도구의 고침(끝 토큰 걸음, BOS 토큰, 문맥 길이, 결정성 재실행)은 그 실행이 끝난 뒤이자 V(12:19 부터) · L(14:51 부터) 실행 전인 10:44 에 커밋했다. 12:19 전에는 L 도 V 도 자료가 없었으므로 판정기와 도구를 고칠 때 1차 비교를 들여다볼 수는 없었다. 등록은 독립된 등록소가 아니라 저자의 저장소에 있고, 등록 · 실행 · 분석 · 이 글까지 하루 안에 끝났다. 다른 점은 부록 B 에 적었다.

5 결과

5.1 타당성 확인

- **같은 입력**: 모든 갈래 · 문항에서 엔진이 받은 프롬프트 토큰 수가 보낸 수와 같다.
- **결정성**: 질문 50 개를 동시 요청 1 로 다시 돌리면 G 와 V 에서 토큰이 같고, 억지로 넣은 토큰의 이어 쓰기 50 개를 다시 돌려도 토큰이 같다.
- **L 의 측정 길**: 슬롯 하나의 `llama-server` 가 생성한 질문 50 개에서, 따라가기의 1위가 G 와 처음 갈라지기 전의 3,358 걸음(문항당 평균 67) 모두에서 서버의 토큰과 같다.
- **증상 판정기의 사람 확인**(증상마다 20 항목: 양성 판정 10 · 음성 판정 10): 반복 19/20, 다른 문자 17/20 — 미리 등록한 90 % 미달이고, 20 항목이라 85 % 일치의 95 % 구간은 약 62–97 % 다. 다른 문자 판정기는 한자말을 설명하며 쓴 한자와 질문에서 인용한 영어 노래 제목을 증상으로 세었다. 비교마다 두 갈래가 같은 판정기를 쓰므로 짝 검정은 어느 쪽으로도 치우치지 않는다. 더 엄격한 사후 판정기(5.6절)로도 모든 결론이 그대로다.

5.2 궤적: 다시 굴렸고, 늘었다고 잡히지는 않는다

표 2 가 1차 시험이다. 둘 다 유의하지 않았고, 구간은 어느 쪽으로든 큰 효과는 배제하지만 보통 크기의 증가는 배제하지 못한다: 자료는 llama.cpp 가 다른 문자의 답을 5분의 1 쯤 더 만드는 것과도, 6분의 1 쯤 없애는 것과도 어긋나지 않는다. llama.cpp 는 LCB 질문의 89 %, 긴 글 300 개 모두, 수학 문제의 97 % 에서 다른 글을 냈다. 첫 갈림은 중앙값으로 66 · 57 · 138 토큰 뒤에 왔다. 손실 하나인 V 는 더 많이 바꾸었고(96–100 %, 첫 갈림 중앙값 21–61 토큰) 마찬가지로 균형이 맞았다 (H1′: 생김 48 · 사라짐 43; H2′: 25 · 28).

H2 는 끝내지 못한 답을 틀림으로 세는데, 틀린 답의 대부분이 끝내지 못한 것이었다: G 의 80 중 59, L 의 77 중 53 이 boxed 답 없이 3,200 토큰 한도에 닿았다(생김 29 · 사라짐 35). 끝낸 답 가운데 틀린 것은 G 21 · L 24(생김 13 · 사라짐 10, $p = 0.68$)였다. V 는 더 자주 끝냈고(끝내지 못함 48) 끝낸 답 가운데 더 자주 틀렸다(29; 생김 17 · 사라짐 9, $p = 0.17$). 보조 비교는 어느 것도 유의하지 않았다: 깨진 옮겨 쓰기 생김 4 · 사라짐 3, 끝내지 못한 수학 29 · 35, 긴 글의 끝없는 반복 19 · 15(모두 $p \geq 0.25$). 마지막 비교는 생김 + 사라짐 = 34 로 사전 등록이 판정에 요구한 40 에

표 3: Qwen2.5-7B 의 위험한 걸음에서의 기울기(출력 972,380 토큰). $\bar{\Delta}$: 평균 기울기, nat(식 1); 구간: 질문 단위 붓스트랩의 95 % 구간; 대조: 같은 걸음에서 가장 높은 정상 대안에 대한 같은 양; 마지막 줄: llama.cpp 자신이 b 를 고른 걸음.

	O: 다른 문자	R: 되풀이	C: 글자 조각
위험한 걸음	3,042	4,975	2,800
천 토큰마다	3.13	5.12	2.88
$\bar{\Delta}_L$	−0.00006	**+0.0039**	−0.0034
95 % 구간	[−0.0046, +0.0046]	[**+0.0015**, **+0.0063**]	[−0.0074, +0.0004]
Holm p	0.99	**0.003**	0.18
판정	기울지 않는다	**나쁜 쪽으로 기운다**	기울지 않는다
대조 $\bar{\Delta}_L$	+0.0010	−0.0011	+0.008 (103 걸음)
$\bar{\Delta}_V$	−0.018	+0.012	+0.012
95 % 구간	[−0.031, −0.006]	[+0.005, +0.018]	[+0.002, +0.023]
L 이 b 를 고름	37 (1.2 %)	101 (2.0 %)	55 (2.0 %)

표 4: 계약대로 정확한 계산(갈래 G)에서 Qwen2.5-7B 의 증상 수.

묶음	문항	S1 반복	S2 다른 문자	S6 깨진 옮겨 쓰기
자연: 한국어 LCB	396	6	65 (16 %)	–
자연: 한국어 긴 글	300	95 (32 %)	29	–
방아쇠: 희귀 음절	200	10	44	115 (58 %)
방아쇠: 섞인 문자	200	6	80 (40 %)[a]	–
방아쇠: 되풀이	200	52 (26 %)	1	–

[a] 5.6절의 엄격한 판정기로는 36 (18 %).

못 미치므로, 기울기가 잡힌 유일한 증상(5.3절)이 궤적 시험에서는 검정력이 가장 낮은 증상이기도 하다 — 방향은 양(+)이다. 수학 600 문항의 정답률은 86.7 %(G) · 87.2 %(L)였다.

5.3 기울기: 되풀이 쪽으로만, 작게

되풀이의 기울기(표 3)는 되풀이 토큰의 승산이 0.39 % 오르는 크기다. 같은 걸음의 대조는 0 이라, 이 이동은 분포가 전체적으로 평평해진 것이 아니라 되풀이 토큰에 특별하다. llama.cpp 가 비정상 토큰을 고른 곳은 모두 여유가 0.5 nat 미만이었다(0.25 nat 미만에서 O 걸음 206 개 중 36, R 걸음 754 개 중 101, C 걸음 314 개 중 52). 활성값 반올림 간격이 llama.cpp 의 2–4 배인 V(4.1절)는 되풀이 쪽으로 세 배쯤 더 기울고, 바이트 오류 쪽으로도 기울며(llama.cpp 는 여기서 기울지 않는다), 다른 문자 토큰에서는 *거꾸로 멀어진다*: 손실마다 기우는 쪽이 다르고, 되풀이의 기울기는 활성값 반올림이 거칠수록 커진다. 이는 활성값 반올림이 되풀이 기울기의 원천 하나라는 것과 어긋나지 않지만, V 는 llama.cpp 손실의 통제된 복제가 아니라 원천을 가려내지는 못한다. R1-Distill 의 영어 추론(되풀이 걸음 8,121 개)에서는 L 도 V 도 기울지 않는다(−0.0006, −0.0009).

5.4 계약대로 정확한 계산과 탐욕 디코딩에서 실패를 내는 것은 모델이다

R1-Distill 은 수학 문제의 86.7 % 를 맞혔고, 틀린 답 80 개 가운데 59 개는 3,200 토큰 안에 끝내지 못했다. 희귀 음절 방아쇠에서 모델은 때때로 이모지 하나를 끝없이 되풀이하거나, 중국어로 'user' 차례를 지어내고 한 글자를 한도까지 되풀이했다(표 4). 이 비율은 탐욕 디코딩의 것이다 — 탐욕 디코딩은 반복을 잘 만드는 것으로 알려져 있고, 뽑기(샘플링)에서는 달라질 것이다. 실제 한국어 글에서 음절의 11.2 %(Qwen2.5) · 8.5 %(Llama 3.1)가 두 개 이상의 바이트 조각 토큰에 걸쳐 나오고, 한글 음절 11,172 개 가운데 한 토큰인 것은 2,512 개(Qwen2.5) · 599 개(Llama 3.1)뿐이다.

5.5 번짐과 오염

다른 문자 위험한 걸음에서 비정상 토큰을 억지로 넣으면 다음 256 토큰 안에 다른 문자의 글이 또 나오는 경우가 100 개 중 80 개로, 원래 토큰 뒤의 34 개보다 많다(불일치 47 · 1, $p = 3.5 \times 10^{-13}$). 넣은 토큰 100 개 중 99 개가 온전한 글자였고, 이어 쓴 글의 앞 두 글자를 빼고 세어도 99 개 중 77 개라, 바이트 열을 기계적으로 마저 채우는 효과가 아니다. 이 걸음들은 정의상 다른 문자 토큰이 이미 상위 20 안에, 고른 토큰의 $\ln 10$ 안에 든 문맥이다: 이 비교가 보이는 것은 그런 문맥에서 토큰 하나가 글의 흐름을 바꾼다는 것이지, 아무 엇나감이나 그런다는 것이 아니다. 되풀이와 글자 조각 걸음에서는 척도가 천장에 닿았다(원래의 이어 쓰기에 이미 증상이 89 % · 97 % 있었다). 앞 답에 넣은 비정상 하나는 다음 답의 증상을 늘리지 않았다: 다른 문자의 글 32 · 37(생김 5 · 사라짐 10), 깨진 옮겨 쓰기 53 · 54, 되풀이한 문장 7 · 10.

5.6 증상 판정기에 대한 민감도

사람 확인 뒤, 다시 셈하기 전에 정한 더 엄격한 판정기는 따옴표나 괄호 안의 외국 글, 뒤에 한글 읽기가 괄호로 붙은 외국 글, 질문에 나오는 라틴 낱말을 세지 않는다. 사람의 판정과 20 항목 중 19 개에서 같다(이 항목들로 규칙을 정했으므로 독립된 확인은 아니다). 새 표본 40 항목으로 한 확인은 부록 F 에 적는다 *[진행 중]*. 모든 결론이 그대로다: H1 생김 31 · 사라짐 31, H1′ 41 · 43, 번짐 78 % 대 31 %. 비율만 움직인다: 섞인 문자 방아쇠의 다른 문자 비율은 40 % 에서 18 % 로, LCB 는 16 % 에서 15 % 로.

6 논의

망가뜨린다고 잡히지 않는다. 8 비트 가중치를 쓰는 주류 엔진의 정밀도에서 계산 손실은 아슬아슬한 자리에서 다시 뽑기처럼 움직인다: 긴 답은 거의 다 바뀌지만, 새로 만드는 실패는 — 우리 검정력 안에서 — 없애는 실패와 짝을 이룬다. 구간은 보통 크기의 증가 — 다른 문자 증상 수의 5 분의 1 까지 — 를 남겨 두므로, 이 결과는 한계이지 부재의 증명이 아니다. 우리가 볼 수 있는 해는 재현성이지 — 열 문항 중 하나쯤(696 중 71, 600 중 53)의 증상 여부가 엔진에 달린다 — 품질이 아니다.

하나의 기울기. 되풀이의 기울기는 실재하고 특별하며, 활성값 반올림이 더 거친 V 가 그것을 세 배 강하게 보이므로 활성값 반올림이 기여자 하나로 짚이지만 가려내지는 못한다. 승산 0.4 % 의 크기로는 우리 표본에서 궤적 수준까지 닿지 않았고, 되풀이의 궤적 시험은 모든 시험 가운데 검정력이 가장 낮았다(생김 + 사라짐 = 34).

탐욕 디코딩에서 원천은 주로 모델이다. 계약대로 정확한 산술에서도 모델은 긴 한국어 답의 삼분의 일에서 되풀이하고, LCB 답의 육분의 일에서 문자를 섞고, 희귀 음절 옮겨 쓰기의 대부분을 틀린다. 다른 문자 토큰이 이미 아슬아슬한 경쟁자인 자리에서 그것이 나오면 모델은 대개 그 문자로 글을 끌고 간다. 이 실패의 대책은 산술이 아니라 모델과 디코딩 절차에 있다. 탐욕 디코딩은 반복을 잘 만들고, 뽑기에서의 바탕 비율은 여기서 재지 않았다.

기준은 무엇이고 무엇이 아닌가. 계약대로 정확하고 결정적이지만, 정확한 것은 아니다: 활성값을 두 자리로 반올림하고 지수 함수를 다항식으로 계산한다. fp64 와의 차이는 0.5B 에서(그리고 시뮬레이션으로 7B 에서) llama.cpp 의 1,000 분의 1 급이고, 자신의 반올림이 위험한 걸음을 기울이는지는 부록 E 가 다룬다. 이 기준은 0.004 nat 의 이동을 잴 수 있게 했고 모든 사례를 비트까지 다시 열 수 있게 했다. 같은 결과가 사용자에게의 실용적 가치도 한정한다: 그런 기준이 있어야 보일 만큼 작은 효과는 사용자가 보는 것을 바꾸기에는 너무 작다.

한계. 한 계열의 7B 모델 둘, Q8_0, 탐욕 디코딩, 실제 엔진 하나(llama.cpp, 기본 설정), GPU 하나(RTX 4070 Ti), 대부분 한국어 질문, 그리고 엔진 수준의 손실만: 4 비트 키-값 캐시, 4 비트 가중치(Patodiya (2026) 는 거기서 갈림이 더 크다고 보고했다), 더 긴 문맥, 뽑기 같은 더 큰 손실은

시험하지 않았다. 판정기는 자동이다 — 다른 문자 판정기는 사람과 20 항목 중 17 에서 같았고, 엄격한 판정기는 그 항목들로 설계했다. 따라가기는 llama.cpp 의 묶음 산술의 한 실현이다. V 는 llama.cpp 의 활성값 양자화보다 거칠다. 되풀이와 글자 조각 걸음의 번짐은 척도가 천장에 닿아 잴 수 없었다. 닫힌 프런티어 모델은 연구할 수 없었다. 기제(바이트 조각 · 아슬아슬한 자리 · 나온 토큰의 끌림)는 일반적이지만 거기서의 비율은 모른다.

7 맺음말

계약대로 정확하고 잡음 없는 기준과 같은 입력으로 보면, 실제 엔진의 계산 손실은 LLM 의 출력을 다시 굴리되 익숙한 실패를 늘린다고 잡히지 않는다 — 순증가는 기준 쪽 증상 수의 약 ±20 % 안에 묶인다 — 그리고 잴 수 있는 유일한 치우침은 되풀이 쪽으로의 0.4 % 기울기다. 탐욕 디코딩에서 실패는 모델에서 오고, 이미 그럴듯한 자리의 다른 문자 토큰 하나가 글을 끌고 간다. 8 비트 정밀도에서 추론 산술을 계약대로 정확하게 만드는 것은 재현성을 사는 일이지 실패를 줄이는 일이 아니다.

감사의 글과 AI 도움

글(geul) 컴파일러, GeulPU, 실험 코드는 저자가 AI 코딩 에이전트(Anthropic Claude)의 상당한 도움을 받아 개발했고, 이 원고의 초안도 그 도움으로 썼다. 연구의 설계, 사전 등록의 승인, 증상 판정기의 손 확인은 저자가 했다. 초안에 대한 한 독자의 검토가 순증가의 구간, H2 의 분리, 4.6 절의 시각, 제목과 결론의 표현으로 이어졌다.

참고문헌

[초안: 항목은 아직 정리하지 않음 — 영문 초안과 같다.]

Bit-exact AI inference verification without performance tradeoffs. arXiv:2606.00279, 2026. [저자 추가 예정.]

T.-R. Chen. Deterministic LLM inference across GPU kernels: power-of-two INT8 quantization scales and the limits of tolerance-based conformance. arXiv:2609.00363, 2026.

K. Cobbe et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

L. Cooper et al. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. arXiv:2609.25624, 2026.

H. He et al. Defeating nondeterminism in LLM inference. Thinking Machines Lab, 2025.

S. Land and M. Bartolo. Fishing for Magikarp: automatically detecting under-trained tokens in large language models. arXiv:2405.05417, 2024.

K. Marchisio et al. Understanding and mitigating language confusion in LLMs. arXiv:2406.20052, 2024a.

K. Marchisio et al. How does quantization affect multilingual LLMs? arXiv:2407.03211, 2024b.

M. Nasr et al. Scalable extraction of training data from (production) language models. arXiv:2311.17035, 2023.

D. Pape et al. The silent hyperparameter: quantifying the impact of inference backends on LLM reproducibility. arXiv:2605.19537, 2026.

A. Patodiya. Same request, different answer: quantization amplifies cache-induced divergence in LLM serving. arXiv:2609.04748, 2026.

J. Qin et al. Vosti: specifying, implementing and verifying deterministic LLM inference. arXiv:2609.38981, 2026.

S. Woo. Raw data—GeulPU stage 40: engine numerical loss and LLM failure symptoms [data set]. Zenodo, 2026. https://doi.org/10.5281/zenodo.23417759

J. Xu et al. Learning to break the loop: analyzing and mitigating repetitions for neural text generation. arXiv:2206.02369, 2022.

J. Yuan et al. Give me FP32 or give me death? Challenges and solutions for reproducible reasoning. arXiv:2506.09501, 2025.

부록 A GeulPU 의 비트 동일성 증거

한 기계 안에서. CPU 참조 구현과 견주어: Qwen2.5-0.5B 의 모든 로짓 행; Qwen2.5-7B Q8_0 의 로짓 26,155,008 개가 차이 0; Q4_0 의 커널 사례 106 개와 k-quant 의 349 개 그리고 파일 전체; Qwen3 · Llama 3.x · Mistral 파일의 끝에서 끝까지. 실행 길 사이에서: 디코드 그래프, 프롬프트 전체, 네 조각으로 나눈 프롬프트, 한 행씩, 되풀이 실행이 같은 로짓과 키-값 캐시를 낸다. wikitext-2 로그 가능도는 글을 512 · 128 · 1 토큰 덩어리로 넣어도 비트까지 같다(llama.cpp 의 perplexity 는 묶음 설정에 따라 15.8171 과 15.8200 사이를 움직인다). 서버는 모델 4 × 요청 12 × 일정 4 에서 같은 토큰과 로짓 해시를 낸다(llama-server 는 12 사례 중 11 에서 탐욕 답을 바꾸었다). lm-evaluation-harness 의 문항 결과는 동시 요청 1 과 8, 그리고 CUDA 와 Vulkan 에서 같다. Linux(WSL2)의 출력은 Windows 의 출력과 바이트까지 같다.

기계들 사이에서. 교차검증 패키지는 지원자의 PC 에서 컴파일러 · 백엔드 · 서버 · 7B 모듈을 소스에서 빌드하고(빌드 해시를 기준과 견준다), DeepSeek-R1-Distill-Qwen-7B Q8_0 을 받아 합성 수학 32 문제를 탐욕으로 풀고(≤3,200 토큰), 글과 함께 모든 걸음에서 152,064 항목의 로짓 행 전체의 64 비트 FNV 해시를 견준다(표 5).

표 5: 기계들 사이의 교차검증. 글: 기준과 글이 같은 문제 수; 로짓 해시: 기준과 로짓 행의 해시가 같은 걸음 수.

PC	GPU (길)	드라이버	글	로짓 해시
기준	NVIDIA RTX 4070 Ti, Ada (CUDA)	616.56	기준	기준
지원자 1	NVIDIA RTX 5070, Blackwell (CUDA)	596.21	32/32	82,673/82,673
지원자 2	NVIDIA RTX 3080 Ti, Ampere (CUDA)	616.92	32/32	82,673/82,673
기준	RTX 4070 Ti (Vulkan; 커널 변형 둘)	616.56	32/32	82,673/82,673
지원자 3	AMD RX 9060 XT, RDNA4 (Vulkan; 변형 둘)	AMD 26.8.1	32/32	82,673/82,673
지원자 4	AMD RX 9060 XT (Vulkan; 변형 둘)	AMD 25.20.42.14	32/32	82,673/82,673

해시한 걸음 82,673 은 생성 토큰 82,657 과 끝 토큰 걸음 16 이다. 같은 기준 PC 에서 llama-server 는 동시 요청 1 과 8 사이에 32 문제 중 0 개에서 같은 글을 냈고 최종 답 6 개를 바꾸었다. Intel UHD 770 내장 GPU 는 0.5B 모델의 기록을 재현했다(더 큰 모델은 메모리 한계를 넘는다). 각 실행은 그때의 커밋으로 엔진을 빌드했다. 이후의 엔진 변경은 기준 출력을 바꾸지 않았고 기준 PC 에서만 다시 확인했다. 지원자들은 배포된 패키지를 돌린 저자의 지인이지 독립된 재현자가 아니며, 32 문제는 합성 문제다.

부록 B 사전 등록과 다른 점

1. 보조 모델 0.5B 를 뺐다: GeulPU 서버는 헤드 크기 128 의 Qwen2 계열 모델만 받는다.
2. 문맥을 슬롯마다 4,096 토큰으로 정했다(미리 정하지 않았다). 새 토큰 256 개를 남길 수 없을 만큼 긴 LCB 질문 3 개를 모든 갈래에서 뺐고, 다른 3 개는 모든 갈래에서 최대 길이를 808 토큰으로 줄였다.
3. R1-Distill 의 질문은 `llama-server` 의 대화 길이 하듯 BOS 토큰과 함께 토큰화했다(GGUF 가 BOS 를 요구하고 틀 글에는 없다). BOS 없이 한 첫 토큰화는 어떤 측정보다 먼저 버렸다.
4. 아주 긴 LCB 답 두 개의 V 점수 매기기가 실패했다(문맥 가득). 토큰이 V 의 상위 20 밖인 위험한 걸음(17 · 1 · 0)은 Δ_V 에서 빠졌다.
5. 글자 조각 걸음의 번짐은 원문이 있는 옮겨 쓰기 방아쇠에서만 쟀다. 되풀이와 글자 조각 걸음의 번짐은 천장에 닿았다.
6. 오염 대화 8 개에서 맞춤법 넣기가 실패했다(가운데 문장의 한글 음절이 셋 미만). 그 짝은 같은 답이라 셈에 들지 않는다.
7. 선택 버그(문항 번호의 문자열 비교)로 첫 실행에서 결정성과 따라가기 확인의 표본이 비었다. 고친 뒤 그 걸음들을 다시 돌렸고 다른 수치에는 영향이 없다.
8. 다른 문자 판정기가 사람 확인과 미리 등록한 90 % 일치에 이르지 못했다(5.1절, 5.6절).
9. 질문 묶음 · 판정기 · 도구 고침은 첫 실행이 시작된 뒤에 커밋했다(4.6절) — 다만 비교 자료는 아직 없었다. 순증가의 구간(5.2절), H2 의 미완/끝낸 오답 분리, 긴 글 되풀이 비교의 '검정력 부족' 해석은 초안에 대한 독자의 검토 뒤에 더한 것이다. 미리 정한 판정 이름은 표에 그대로 둔다.

부록 C 재현과 자료

- **사전 등록과 기록**(한국어): 저장소의 `문서/40단계.md` — 설계(커밋 `9190f76`, 09:28, 첫 실행 09:51 전), 질문 묶음과 판정기(`d88b6cf`, 10:00, G 의 Qwen2.5 실행 중), 도구 고침(`eedb304`, 10:44, 그 실행 뒤 · V · L 실행 전), 결과(`727a218`), 사람 확인과 엄격한 판정기 분석(`0a5ab97`).
- **질문 묶음**: `시험/질문40/` — 출처, 라이선스(LCB: Apache-2.0; gsm8k: MIT), SHA-256 해시는 `시험/질문40/출처.md`; `재기/질문40.py`(씨앗 40)가 만든다.
- **도구**: `재기/증상40.py`(증상 · 위험한 걸음 판정기, 통계; 단위 시험 93 개), `재기/모음40.py`(llama.cpp 를 통한 토큰화, 세 갈래의 생성, V 의 점수 매기기, 오염, 억지 토큰 이어 쓰기, 결정성 재실행), `재기/따라40.py`(C API 로 llama.cpp 의 분포 따라가기), `재기/셈40.py`(분석), `재기/사후40.py`(엄격한 판정기; 단위 시험 12 개), `공식실행/시험40.sh`(걸음의 차례).
- **명령**: `bash 공식실행/시험40.sh` 에 저장소 · 작업 폴더 · ASCII 모델 폴더 · llama.cpp 폴더를 주면 빌드 → 토큰화 → G → 오염 → V → L → 따라가기 → 확인 → 위험한 걸음 → 억지 토큰 이어 쓰기 → 결정성 → 분석을 돌린다.
- **판과 해시**(`문서/결과/40단계/40_기록.txt`): GeulPU 서버 `010f5dae...`, 커널 생성기 `a304a884...`, 모듈 `qwen2Q8_7b` 와 `qwen2Q8_7b_자리하나` 는 저장소의 모듈 해시 표와 같음; llama.cpp b11669(커밋 `620d2225c`, Windows CUDA 13.4 빌드); 모델 Qwen2.5-7B-Instruct Q8_0(SHA-256 `52c5bf14...`)과 DeepSeek-R1-Distill-Qwen-7B Q8_0(`7307bfbb...`); NVIDIA RTX 4070 Ti, 드라이버 616.56.
- **결과**: `문서/결과/40단계/40_판정.json`(모든 수치), `40_사후_S2엄격.json`(엄격한 판정기), `사람확인.md`(사람 확인).
- **원자료**: Zenodo, DOI 10.5281/zenodo.23417759 (Woo, 2026) — zip 묶음 다섯(압축 1.7 GB, 풀면 4.3 GB): G · V · L 의 모든 생성 토큰과 상위 20 로그 확률, 따라가기의 걸음마다의 로그 확률, 위험한 걸음, 억지 토큰 이어 쓰기, 오염 대화, 보낸 토큰 번호, 실험에 쓴 엔진

실행 파일과 커널, 그리고 형식을 설명한 README 와 SHA-256 목록. G 가 낸 모든 사례는 지원하는 어느 GPU 에서나 CPU 참조 구현에서 비트까지 다시 열 수 있다.

부록 D 글(geul) 언어 · 컴파일러 · 엔진

부록 D.1 글(geul) 언어와 그 컴파일러

글(geul)은 문법이 한국어를 따르는 시스템 프로그래밍 언어다: 동사가 끝에 오고, 인수의 역할은 자리가 아니라 조사로 표시한다. 함수는 역할과 함께 선언하고 — `[문자열 원본을 문자열 대상에 복사]` — 역할로 부른다 — `원본을 대상에 복사하다.`. 조사 을/를(목적), 에(도착지), 에서(출처), 로(수단), 와/과(동반)가 인수를 묶고, 같은 타입의 인수에 조사가 없는 호출은 컴파일 오류다. 타입은 평범하다 — 정수는 64 비트 정수, 짧은실수는 32 비트 실수, `T 참조`는 포인터 — 그리고 메모리는 손으로 관리한다.

컴파일러는 자기 자신을 컴파일한다(self-hosted): 글(geul)로 쓰였고(약 10,000 줄) 자기 자신을 빌드한다. Python 으로 쓴 별도의 참조 구현(약 7,000 줄)은 부트스트랩의 씨앗이자 두 번째 구현으로 둔다. 빌드는 고정점을 확인한다 — 참조 구현으로 빌드한 컴파일러, 그것으로 다시 빌드한 컴파일러, 그 출력으로 또 빌드한 컴파일러가 매번 바이트까지 같다. 사용자는 둘 다 필요 없다: 배포된 컴파일러 `geulc.exe`(글 1.3.1, 2026 년 10 월, MIT 라이선스; 182 KB zip, Windows 용 PyPI 패키지 `geul` 로도 배포)가 아래의 모든 것을 빌드한다. 컴파일러는 완전한 자기 백엔드를 가진다 — 어휘 분석기, 파서, 의미 분석, 타입이 붙은 중간 표현, 인라이너, 선형 스캔 레지스터 할당을 하는 x86-64 코드 생성기, 손으로 쓴 명령어 인코더, PE32+ · ELF64 작성기 — 그리고 C 컴파일러 · 어셈블러 · 링커 · LLVM 을 쓰지 않는다. 이것이 만든 Windows 실행 파일은 `kernel32.dll`(과 명령줄을 위한 `shell32.dll`)만 가져오고, Linux 대상(연구 가지)은 `libc.so.6` 만 필요하다. 빌드는 바이트까지 재현된다.

부록 D.2 글(geul) 소스에서 GPU 명령어까지

GPU 지원은 컴파일러의 연구 가지에 있다. GPU 커널은 보통의 글(geul) 함수다. GPU 기능은 선언된 내장 함수로 닿는다: 스레드 · 블록 번호, 공유 메모리, 장벽, 워프 셔플, 비동기 복사(`cp.async`), 정수 텐서 코어 행렬 곱-누적(`mma.sync`... `m16n8k32`... `s8.s8.s32` 와 `m16n8k16`), `dp4a`, 그리고 소스가 쓴 곳에서만 나오는 융합 곱셈-덧셈. 둘 다 글(geul)로 자체 호스팅 프런트엔드 위에 쓴 두 백엔드가 그런 커널을 옮긴다.

- **PTX 백엔드**(2,574 줄) — NVIDIA 의 문서화된 가상 명령어 집합인 PTX 를 낸다. 모든 부동소수점 연산에 반올림 모드(`.rn`)를 명시하므로 드라이버가 곱셈과 덧셈을 제멋대로 융합할 수 없다. 근사 명령어와, 덧셈의 반올림을 PTX 명세가 열어 둔 부동소수점 텐서 코어는 쓰지 않는다.
- **SPIR-V 백엔드**(4,006 줄) — Vulkan(AMD · Intel · NVIDIA)을 위한 SPIR-V 1.6 컴퓨트 셰이더를 낸다. 커널 인수는 푸시 상수가 되고 전역 메모리는 64 비트 장치 주소가 된다. 나눗셈과 제곱근은 바르게 반올림되도록 풀어 쓰고, 정수 행렬 곱은 장치가 제공하면 협력 행렬 명령어를, 아니면 서브그룹 연산을 쓴다.

실행 시 엔진은 PTX 글을 NVIDIA 드라이버에 넘기고(`cuModuleLoadData`), 드라이버에 내장된 JIT 컴파일러가 그것을 GPU 의 기계어(SASS)로 옮기며, 커널은 `cuLaunchKernel` 과 CUDA 그래프로 띄운다(그림 1). Vulkan 에서는 셰이더 모듈과 컴퓨트 파이프라인을 만들고 디스패치를 기록한다. 얇은 층이 엔진이 쓰는 드라이버 API 호출을 Vulkan 으로 대응시키므로 엔진 코드는 공유된다. CUDA 툴킷의 어느 것도 — `nvcc`, `ptxas`, `cudart`, cuBLAS — Vulkan SDK 의 어느 것도 엔진을 빌드하거나 돌리는 데 쓰이지 않는다. CI 가 `ptxas` 를 설치하는 것은 레지스터 넘침을 검사하기 위해서뿐이다. 이것은 다른 엔진이 GPU 와 만나는 것과 같은 층이다: llama.cpp 는 `nvcc` 로 미리 컴파일한 커널을 싣고 cuBLAS 를 부르며, vLLM 은 PyTorch · CUDA · Triton 커널을 돌린다. GeulPU 는 자기 커널을 스스로 만들고 드라이버하고만 말한다. PTX 와 SPIR-V 아래의 제조사 비공개 컴파일러는 계약 밖이다 — 장치 사이의 비트 동일성을 증명이 아니라 실측으로 세우는 까닭이 이것이다(부록 A).

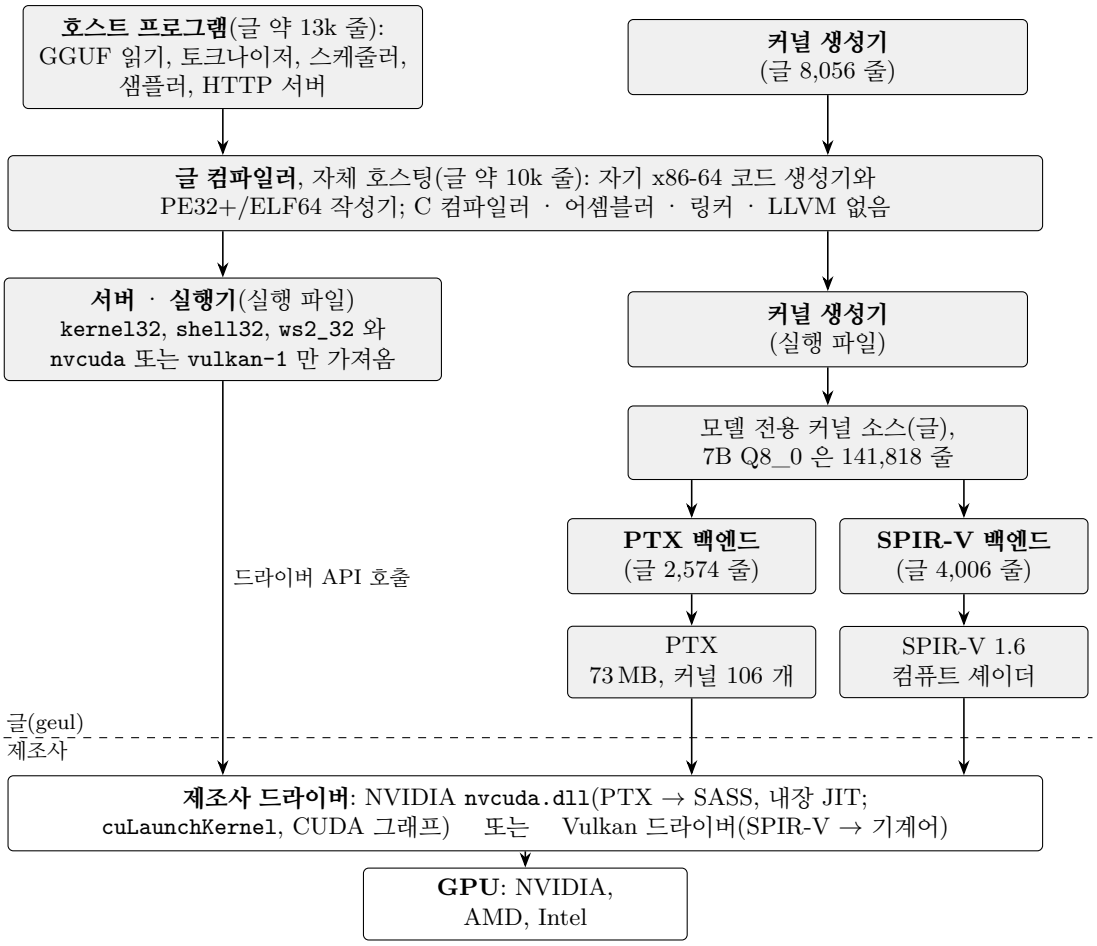


그림 1: GeulPU 의 구조. 점선 위는 모두 글(geul)로 쓰였고 글 컴파일러가 컴파일한다. 점선 아래는 GPU 제조사의 드라이버다.

부록 D.3 커널 생성기와 엔진

커널 생성기는 그 자체가 글(geul) 프로그램이고(8,056 줄), 모델의 모양과 가중치 형식에 맞춘 커널 소스를 쓴다. Qwen2.5-7B Q8_0 에는 141,818 줄의 글을 쓰고, PTX 백엔드가 이것을 커널 106 개와 텐서 코어 명령어 6,488 개가 든 73 MB 의 PTX 로 바꾼다. 모듈은 사용자의 기계에서 처음 쓸 때 빌드되고, CI 는 그런 모듈 18 개를 다시 만들어 SHA-256 해시를 기록된 값과 견준다.

호스트 — 약 13,000 줄의 글(geul)(시험 프로그램까지 15,000 줄) — 는 완전한 서빙 엔진이다.

- **모델**: Q8_0 · Q4_0 · Q4_K · Q5_K · Q6_K 가중치(Q4_K_M · Q5_K_M 혼합 포함)의 GGUF 파일을 읽는다. 구조는 qwen2 · qwen3 · llama — Qwen2.5, Qwen3, DeepSeek-R1-Distill-Qwen, Llama 3.1 과 3.2, Mistral v0.3 — 헤드 크기 128(Qwen2.5-0.5B 는 명령줄 실행기로만).
- **글자**: 유니코드 NFC 를 갖춘 바이트 수준 BPE, Llama 3 BPE, SentencePiece 토크나이저를 글(geul)로 썼고, 시험 글 2,000 줄에서 llama.cpp 와 토큰까지 같다. 대화 틀은 GGUF 의 Jinja 틀과 대조했다.
- **서빙**: OpenAI 호환 HTTP 서버(채팅 · 텍스트 완성, 스트리밍, `/tokenize`), 연속 묶음과 슬롯마다의 프롬프트 재사용을 갖춘 최대 8 슬롯, CUDA 그래프 하나로 잡은 디코드 걸음, 같은 적힌 계약으로 계산하는 샘플링과 로그 확률.
- **발자국**: CUDA 서버는 `kernel32` · `shell32` · `ws2_32` · `nvcuda`(드라이버 API 함수 43 개)만 가져오고, Vulkan 서버는 `nvcuda` 대신 `vulkan-1`(함수 48 개)을 가져온다. CI 가 이 가져오기 표를 검사한다.

RTX 4070 Ti 에서 디코딩 속도는 Qwen2.5-7B 와 Llama-3.1-8B 의 Q8_0 · Q4_K_M 에서 llama.cpp 의 ±5 % 안이고(토큰당 시간 0.97–1.05 배, llama.cpp 판과 문맥 길이에 따라), 프롬프트 처리는 약 128 토큰 아래에서는 llama.cpp 보다 빠르고 8,192 토큰에서는 최대 1.3–1.4 배(Q8_0) · 1.4–1.6 배(k-quant 형식) 느리다. 모든 것이 글(geul)은 아니다: 계약의 CPU 참조 구현, 시험 도구, 벤치마크는 Python(NumPy)이고, 선택 사항인 데스크톱 앱의 화면은 JavaScript 다. 엔진은 Windows 가 먼저이고(Linux 빌드는 WSL2 에서 시험), GPU 하나를 쓰며, CUDA 길에서는 컴퓨트 능력 8.0 이상이 필요하다.

부록 E 기준 자체가 기우는가? (Qwen2.5-0.5B, fp64 대비)

[측정 중. 설계: 7B G 의 답을 접두로 삼아 Qwen2.5-0.5B 의 계약대로 정확한 분포를 GeulPU 엔진으로 걸음마다 계산하고, 4.4절처럼 위험한 걸음을 고르고(0.5B 의 1위가 글의 다음 토큰과 같은 걸음에서), 같은 걸음을 계약의 반올림이 없는 fp64 NumPy 구현으로 다시 계산하여 $\Delta_G = [\ln p_G(b) - \ln p_G(a)] - [\ln p_{64}(b) - \ln p_{64}(a)]$ *를 붓스트랩 구간 · 정상 대안의 대조와 함께 4.4절의 판정 이름으로 적는다.]*

부록 F 엄격한 판정기의 2차 사람 확인

[진행 중: 5.6절의 엄격한 판정기가 양성 · 음성으로 판정한 새 항목 40 개(규칙을 정하는 데 쓴 20 개와 겹치지 않음)를 손으로 판정한다.]